تقرير العمل التطبيقي (TP) - نماذج التصميم

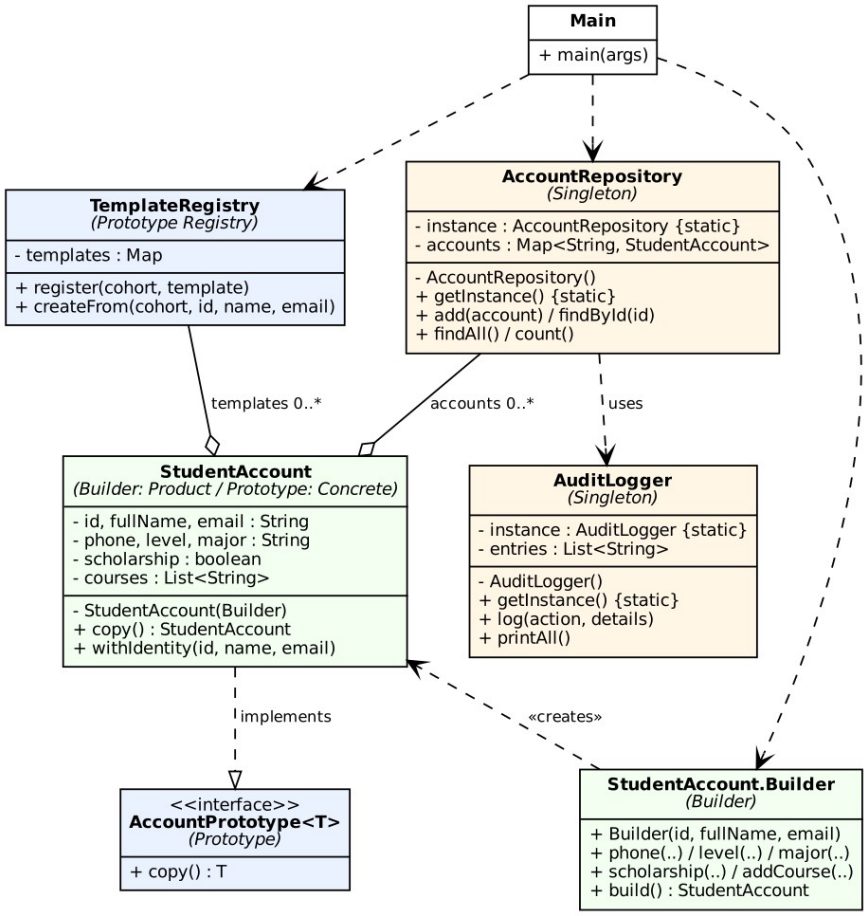
مشروع UniAccounts: نظام إدارة حسابات الطلبة - العمل الأول: النواة وأنماط الإنشاء

الاسم واللقب: ..... الفوج: ..... التاريخ: 10/11/2026

1. مقدمة

في هذا العمل بنيت نواة نظام UniAccounts باستعمال ثلاثة أنماط إنشائية: **Builder** لبناء حساب الطالب، و**Singleton** لضمان وجود مستودع واحد وسجل تدقيق واحد، و**Prototype** لإنشاء حسابات جديدة بنسخ قالب جاهز لكل دفعة. الشيفرة مقسمة إلى حزم (model, prototype, repository, audit) وكل صنف في ملف منفصل، وعند كل نمط تعليق يذكر اسم النمط ودور الصنف فيه.

2. مخطط الأصناف (UML)



الأخضر: Builder - الأزرق: Prototype - البرتقالي: Singleton

3. الأنماط المستعملة

3.1 نمط Builder (البنّاء)

أين استعملته؟ في StudentAccount.Builder وهو صنف متداخل داخل StudentAccount. الحقول الإجبارية الثلاثة (id, fullName, email) تُعطى لمُنشئ الـ Builder، والحقول الاختيارية (phone, level, major, scholarship, courses) تُضبط بدوال متسلسلة، ثم تتحقق الدالة build() من الشروط (id غير

فارغ، البريد يحتوي @، المستوى من L1 إلى M2) وتنشئ الحساب. مُنشئ StudentAccount خاص (private).

ما المشكلة التي حلها؟ للحساب ثمانية حقول بعضها اختياري، فصار الإنشاء مقروءاً مثل level("M1").scholarship(true). بدل مُنشئ طويل. القيم الافتراضية (L1 و false و null وقائمة فارغة) تُطبَّق تلقائياً، ولا يمكن إنشاء حساب غير صالح.

ماذا لو لم أستعمله؟ كنت سأحتاج مُنشئاً واحداً بثمانية وسائط، وسيسهل الخطأ في ترتيبها (مثلاً وضع البريد مكان الاسم) دون أن ينبّه المترجم لأن أغلبها من نوع String، وتصبح الشيفرة غير مقروءة.

3.2 نمط Singleton (الوحيد)

أين استعملته؟ في صنفين: AccountRepository (المستودع المركزي) وAuditLogger (سجل التدقيق). لكل منهما مُنشئ خاص ومتغير ساكن instance والدالة getInstance() (متزامنة synchronized) التي تنشئ الكائن عند أول استعمال وتعيد نفس الكائن بعد ذلك. المستودع نفسه يكتب في سجل التدقيق عند كل إضافة أو رفض.

ما المشكلة التي حلها؟ كل أجزاء البرنامج ترى نفس المستودع ونفس السجل. في العرض التجريبي: repo == repo2 وlog1 == log2 تعطيان true، ومحاولة إضافة معرّف مكرر رُفضت لأن المستودع واحد ويعرف كل الحسابات.

ماذا لو لم أستعمله؟ لو أنشأ كل جزء مستودعه الخاص بـ new لضاعت الحسابات بين النسخ، ولم يمكن ضمان أن رقم التسجيل فريد، وتفرّقت عمليات التدقيق في عدة سجلات.

3.3 نمط Prototype (النموذج الأولي)

أين استعملته؟ الواجهة AccountPrototype تعرّف الدالة copy()، والصنف StudentAccount ينفذها (نسخة عميقة لقائمة المقررات، ولا يُنسخ رقم الهاتف لأنه معلومة شخصية). الصنف TemplateRegistry يحتفظ بقالب لكل دفعة (مثل L2-INFO) والدالة createFrom() تنسخ القالب ثم تغيّر هوية الطالب فقط عبر withIdentity().

ما المشكلة التي حلها؟ مئات الطلبة يلتحقون بنفس التخصص والمستوى ونفس المقررات، فتُنشئ القالب مرة واحدة ثم ننسخه. في العرض: بعد إضافة مقرر English إلى الحساب b فقط، بقي القالب والحساب a دون تغيير، مما يثبت أن النسخ مستقلة.

ماذا لو لم أستعمله؟ كنت سأعيد بناء كل حساب من الصفر وأكرر المستوى والتخصص والمقررات مئات المرات (تكرار وأخطاء). ولو نسخت بالإسناد البسيط لتشاركت الحسابات نفس قائمة المقررات، وتعديل حساب واحد يغيّر الجميع.

3.4 جدول ربط الأنماط بالشيفرة

النمط	الصنف / الأصناف في الشيفرة
Builder	StudentAccount.Builder
Singleton	AccountRepository, AuditLogger
Prototype	AccountPrototype, StudentAccount.copy(), TemplateRegistry

4. لقطات شاشة لنتيجة التشغيل

أ) نمط Builder: إنشاء حساب بالحقول الإجبارية فقط، وآخر بكل الحقول، ورفض بريد بدون @.

```
java -cp out com.uniaccounts.Main

===== 1) BUILDER =====
StudentAccount{id='20240001', name='Amine Haddad', email='amine.haddad@univ.dz', phone=null, level=L1,
    major=null, scholarship=false, courses=[]}
StudentAccount{id='20240002', name='Sara Meziane', email='sara.meziane@univ.dz', phone=0550123456, level=M1
    major=Computer Science, scholarship=true, courses=[Design Patterns, Databases]}
Invalid account refused: email must contain the @ symbol
```

ب) نمط Singleton: المستودع والسجل نسخة واحدة، ورفض معرّف مكرر.

```
java -cp out com.uniaccounts.Main

===== 2) SINGLETON =====
repo == repo2 ? true
log1 == log2  ? true
Duplicate refused: id already exists: 20240001
```

ج) نمط Prototype: إنشاء حسابات من قالب L2، واستقلال النسخ عن القالب.

```
java -cp out com.uniaccounts.Main

===== 3) PROTOTYPE =====
StudentAccount{id='20240101', name='Yacine Bensalem', email='yacine@univ.dz', phone=null, level=L2,
    major=Computer Science, scholarship=false, courses=[Algorithms, Java OOP, Databases]}
StudentAccount{id='20240102', name='Nadia Ouali', email='nadia@univ.dz', phone=null, level=L2, major=Comput
    Science, scholarship=false, courses=[Algorithms, Java OOP, Databases]}
After adding a course to b only:
  template courses = [Algorithms, Java OOP, Databases]
  a courses        = [Algorithms, Java OOP, Databases]
  b courses        = [Algorithms, Java OOP, Databases, English]
```

د) الحالة النهائية للمستودع وسجل التدقيق.

```
java -cp out com.uniaccounts.Main

===== Repository: 5 accounts =====
20240001 | Amine Haddad | L1 | null
20240002 | Sara Meziane | M1 | Computer Science
20240101 | Yacine Bensalem | L2 | Computer Science
20240102 | Nadia Ouali | L2 | Computer Science
20240103 | Lina Kaci | L2 | Computer Science

===== Audit log =====
[2026-11-10 20:28:11] CREATED - account 20240001 (Amine Haddad)
[2026-11-10 20:28:11] CREATED - account 20240002 (Sara Meziane)
[2026-11-10 20:28:11] REJECTED - duplicate id 20240001
[2026-11-10 20:28:11] CREATED - account 20240101 (Yacine Bensalem)
[2026-11-10 20:28:11] CREATED - account 20240102 (Nadia Ouali)
[2026-11-10 20:28:11] CREATED - account 20240103 (Lina Kaci)
```